

A module is a Python file (.py) containing functions, variables, classes, or other Python code that can be reused in another program.

For example, create:
calculator.py

Why use modules?

Without modules:

- main.py
 - +---- Hundreds/thousands of lines
 - +---- Difficult to maintain

With modules:

- main.py
 - +---- calculator.py
 - +---- student.py
 - +---- employee.py
- Each file handles a specific responsibility.

Why Modules are Used?

Modules provide:

- +---- Code reusability
- +---- Modularity
- +---- Easy maintenance
- +---- Better organization
- +---- Namespace management
- +---- Avoiding duplicate code

Built-in Modules

- +---- Python provides many modules as part of the standard library.

Module Purpose

- Math : Mathematical operations
- random : Random number generation
- statistics : Statistical calculations
- datetime : Date and time

os : Operating system operations
sys : Python interpreter/system information
calendar : Calendar operations
collections : Specialized data structures
re : Regular expressions
json : JSON data processing

Import Statements

There are several ways to import functionality.

+---- Import the Entire Module

```
import math  
print(math.sqrt(25))  
print(math.pi)
```

Here: math is the module and sqrt(), pi are members of the module.

+---- Import a Specific Function

Instead of importing the entire module:

```
from math import sqrt  
print(sqrt(25))
```

Now we don't need: math.sqrt()

+---- Import Multiple Functions

```
from math import sqrt, factorial  
print(sqrt(49))  
print(factorial(5))
```

+---- Import Using an Alias

An alias gives a module a shorter name.

```
import math as m  
print(m.sqrt(64))  
print(m.pi)
```

Another example:

```
import statistics as stats
marks = [70, 80, 90, 85, 75]
print(stats.mean(marks))
```

Packages

A **package** is a directory containing related Python modules.

For example:

```
college/
|
├── __init__.py
├── student.py
├── faculty.py
└── marks.py
```

The package can contain multiple modules:

```
college
|
+--- student.py
+--- faculty.py
+--- marks.py
```

This becomes useful when a project grows.

Standard (collections, functools, itertools, math, datetime, random, sys)

Python's standard library includes these modules pre-installed to handle specialized containers, functional operations, loops, math, time, randomness, and system parameters.

Core Standard Library Modules

- **collections:** Provides specialized container datatypes like Counter, namedtuple, defaultdict, and deque as efficient alternatives to standard dicts and lists.
- **functools:** Offers higher-order functions and operations on callable objects, such as functools.reduce() or lru_cache.

- **itertools:** Implements fast, memory-efficient iterator building blocks like `permutations()`, `combinations()`, and `product()`.
- **math:** Gives access to mathematical constants like `pi` and functions like `sqrt()` and `factorial()`.
- **datetime:** Supplies classes for manipulating dates and times, supporting `datetime.now()` and custom date objects.
- **random:** Generates pseudo-random numbers, shuffles sequences, and handles data sampling (avoid for security needs).
- **sys:** Provides access to system-specific parameters, functions, and standard I/O streams like `sys.stdout` and `sys.argv`

collections Module

The collections module provides specialized container data types.

Important functions:

- `Counter`
- `defaultdict`
- `deque`
- `namedtuple`

1. Counter

A Counter is a dict subclass designed to count the frequencies of hashable elements. Since it inherits from dict, it supports all standard dictionary methods (like `.keys()`, `.values()`, and `.items()`) except for `.fromkeys()`.

Specialized Methods

- **`.most_common([n])`:** Returns a list of the n most common elements and their counts, ordered from most frequent to least. If n is omitted, it returns all elements.
- **`.elements()`:** Returns an iterator over the elements, repeating each element as many times as its count. Elements with a count of zero or less are ignored.
- **`.subtract([iterable_or_mapping])`:** Subtracts element counts based on an iterable or another mapping/counter. Counts can fall below zero.

- **.update([iterable_or_mapping]):** Adds or adds-in element counts from an iterable or another mapping/counter (instead of replacing them like a standard dictionary).
- **.total():** Returns the sum of all element counts.

2. defaultdict

A defaultdict is a dict subclass that overrides one method and adds one writable instance variable to automatically supply values for missing keys. It inherits all other standard dictionary methods.

Specialized Methods & Attributes

- **default_factory:** An attribute/argument that holds a callable function (like int, list, or set) which provides the default value when a requested key is missing.
- **.__missing__(key):** This internal magic method is automatically triggered by dict.__getitem__ when a key is not found. It calls default_factory to create a default entry for that key.

3. deque

A deque (double-ended queue) is a list-like container optimized for O(1) fast, memory-efficient appends and pops from both ends.

Specialized Methods

- **.append(x):** Adds x to the right side of the deque.
- **.appendleft(x):** Adds x to the left side of the deque.
- **.pop():** Removes and returns an element from the right side.
- **.popleft():** Removes and returns an element from the left side.
- **.extend(iterable):** Appends elements from the iterable to the right side.
- **.extendleft(iterable):** Appends elements from the iterable to the left side (reversing their original order).
- **.rotate(n=1):** Rotates the deque n steps to the right. If n is negative, it rotates to the left.
- **.maxlen:** A read-only attribute returning the maximum size of the deque (or None if unbounded). When a bounded deque is full, adding items on one side causes items to drop off the opposite side.

- **Other standard sequence methods:** `.clear()`, `.copy()`, `.count()`, `.index()`, `.insert()`, `.remove()`, and `.reverse()`

functools Module

`functools` provides functions that work with other functions.

Important functions:

- `reduce()`
- `partial()`
- `lru_cache()`
- `wraps()`

`reduce()`: takes a function and a sequence, and applies that function cumulatively to the items to reduce the sequence to a single value.

`partial()`: allows you to freeze a portion of a function's arguments and/or keywords, creating a new, simplified function.

`lru_cache()`: wraps a function with a Least Recently Used (LRU) cache, which saves the results of function calls.

`itertools` Module: `itertools` provides tools for efficient iteration.

Important functions:

- `count()`
- `cycle()`
- `repeat()`
- `chain()`
- `combinations()`
- `permutations()`
- `product()`

1. Infinite Iterators

These functions produce sequences that will loop indefinitely unless broken by a condition (like a `break` statement or a loop limit).

- **count(start=0, step=1)**: Generates an infinite, evenly spaced sequence of numbers.
 - *Example*: `count(10)` yields 10, 11, 12, 13, ...
- **cycle(iterable)**: Saves a copy of the elements from an iterable and repeats them endlessly in order.
 - *Example*: `cycle('ABC')` yields A, B, C, A, B, C, ...
- **repeat(object, times=None)**: Streams the exact same object over and over again. If times is specified, it stops after that many repetitions.
 - *Example*: `repeat('Hello', 3)` yields 'Hello', 'Hello', 'Hello'

2. Combinatoric Iterators

These functions are used to find mathematical arrangements, combinations, and permutations of data. They are incredibly useful for probability, statistics, and puzzle-solving algorithms.

- **product(*iterables, repeat=1)**: Computes the **Cartesian product** of the input iterables. It is the equivalent of nested for-loops.
 - *Example*: `product([1, 2], ['A', 'B'])` yields (1, 'A'), (1, 'B'), (2, 'A'), (2, 'B')
- **permutations(iterable, r=None)**: Returns successive **r-length permutations** (ordered arrangements) of elements. Elements are treated as unique based on their position, not their value.
 - *Example*: `permutations('ABC', 2)` yields ('A', 'B'), ('A', 'C'), ('B', 'A'), ('B', 'C'), ('C', 'A'), ('C', 'B')
- **combinations(iterable, r)**: Returns successive **r-length combinations** (unordered groups) of elements. Unlike permutations, the order of elements inside the tuple doesn't matter, and elements are not repeated.
 - *Example*: `combinations('ABC', 2)` yields ('A', 'B'), ('A', 'C'), ('B', 'C')

3. Flattening Iterators

- **chain(*iterables)**: Groups multiple iterables together into a single, seamless sequence. It treats them as if they were one giant list, without actually creating a new list in memory.
 - *Example*: `chain([1, 2], [3, 4])` yields 1, 2, 3, 4

math Module:

The math module provides mathematical functions.

Important functions:

sqrt()
factorial()
ceil()
floor()
gcd()
lcm()
pow()
log()
sin()
cos()
radians()

datetime Module:

provides the fundamental classes for managing dates, times, and intervals in Python. It handles everything from calendar calculations to time zone adjustments and string parsing.

Used for working with:

- Dates
- Time
- Date and time
- Time differences
- Formatting

1. Working with Dates (date class)

The date class handles standard calendar dates (Year, Month, Day) using the Gregorian calendar.

- **date.today():** Returns the current local date.
- **Attributes:** .year, .month, and .day.
- **.weekday():** Returns the day of the week as an integer (Monday is 0, Sunday is 6).

2. Working with Time (time class)

The time class represents an idealized time of day, independent of any particular date.

- **Attributes:** `.hour`, `.minute`, `.second`, `.microsecond`, and `.tzinfo` (for time zones).
- It is purely for storing clock times (e.g., setting a daily alarm at 08:30 AM).

3. Combining Date and Time (datetime class)

The datetime class combines all attributes of both the date and time classes together.

- **datetime.now():** Returns the current local date and time.
- **datetime.utcnow():** Use `datetime.now(timezone.utc)` instead to get the current UTC time.
- **.combine(date, time):** Merges a separate date object and time object into a single datetime instance.

4. Formatting & Parsing (Strings vs. Objects)

To convert dates between human-readable strings and Python objects, use the "format" and "parse" methods.

- **strftime() (String Format Time):** Converts a datetime object into a String.
- **strptime() (String Parse Time):** Parses a String into a datetime object.

random Module:

The random module generates pseudo-random values.

Important functions:

- `random()`
- `randint()`
- `randrange()`
- `choice()`
- `choices()`
- `sample()`
- `shuffle()`
- `uniform()`

sys Module:

provides access to Python interpreter/system-specific information.

Important features:

- `sys.argv`
- `sys.version`
- `sys.platform`
- `sys.path`
- `sys.exit()`
- `sys.getsizeof()`

1. Command-Line Arguments (`sys.argv`)

`sys.argv` is a **list of command-line arguments** passed to a Python script.

- `sys.argv[0]` is always the name of the script itself.
- `sys.argv[1:]` contains the actual arguments provided by the user.

```
import sys
```

```
# If run as: python script.py file.txt 100
```

```
print(sys.argv) # Output: ['script.py', 'file.txt', '100']
```

2. Runtime Information (`sys.version` & `sys.platform`)

These variables give you insight into the environment running your code, which is essential for writing cross-platform compatible scripts.

- **`sys.version`:** A string containing the version number of the Python interpreter plus additional build information.
- **`sys.platform`:** An identifier for the host operating system platform (e.g., 'win32' for Windows, 'linux' for Linux, or 'darwin' for macOS).

3. Modifying Module Search Paths (`sys.path`)

`sys.path` is a **list of strings** that specifies the search path for modules. When you run `import abc`, Python searches directories in this exact order.

- You can dynamically append new paths to this list (`sys.path.append('/path/to/module')`) to import files from custom directories.

4. Direct Operations (`sys.exit()` & `sys.getsizeof()`)

- **`sys.exit([arg])`:** Exits the Python interpreter. It raises a `SystemExit` exception, allowing cleanup code in finally blocks to run. Passing an integer (like `sys.exit(0)`) signals a successful termination to the OS, while any non-zero value signals an error.

- **sys.getsizeof(object)**: Returns the **exact memory size of an object in bytes**. It accounts for the internal garbage collection overhead of Python objects.

```
import sys
x = [1, 2, 3]
print(sys.getsizeof(x)) # Output: Memory footprint in bytes (varies by system)
```

Creating a Package

Create a **Python package** in VS Code, use the following simple procedure. Let's create the university package from your example.

1. Create a Project Folder

In VS Code, create a folder such as:

UniversityProject

Open this folder in VS Code.

Your initial structure will be:

UniversityProject/

2. Create a Package Folder

Inside UniversityProject, create a new folder called:

university

So:

UniversityProject/

└── university/

3. Create `__init__.py`

Inside the university folder, create a Python file:

`__init__.py`

Your structure becomes:

UniversityProject/

└── university/

 └── `__init__.py`

For a basic package, `__init__.py` can be empty.

4. Create Modules Inside the Package

Now create:

student.py

result.py

The final structure is:

UniversityProject/

```
|
|—— university/
|   |—— __init__.py
|   |—— student.py
|   |—— result.py
|
|—— main.py
```

Suppose we create:

university/

```
|
|—— __init__.py
|—— student.py
|—— result.py
```

student.py

```
def student_details(name, course):
```

```
    print("Name:", name)
```

```
    print("Course:", course)
```

result.py

```
def calculate_percentage(marks):
```

```
    return sum(marks) / len(marks)
```

Main program

```
from university.student import student_details
```

```
from university.result import calculate_percentage
```

```
student_details("Anita", "MCA")
```

```
marks = [85, 90, 78, 88]
```

```
print("Percentage:",
```

```
      calculate_percentage(marks))
```

17. Package Structure

Students should understand the relationship:

Package

```
|
+---- Module 1
|
+---- Module 2
|
+---- Module 3
```

For example:

university/

```
|
|—— __init__.py
|
|—— student.py
|   |—— student_details() #methods
|   |—— student_search.py #sub-modules
|
|—— marks.py
|   |—— total()
|   |—— percentage()
|
|—— attendance.py
|   |—— present()
|   |—— absent()
```